

RAGEN-Ward: Reproducing Multi-Turn LLM Agent Reinforcement Learning on Consumer Hardware and Probing the Algorithmic Boundary of Variance Filtering

WANG Jianyang

Student ID: 21191450

HKUST Independent Project

Repository: https://github.com/KyleWard-personal-organization/ragen_ward

Abstract

This report presents RAGEN-Ward, a consumer-hardware reproduction and extension of RAGEN, a recent framework for reinforcement learning in multi-turn LLM agent environments. Under a single NVIDIA RTX 4070 with 8 GB VRAM and a Windows software stack, the project rewrites the original RAGEN-style pipeline into five modular components, implements a memory and throughput optimization stack for 0.5B-scale LLM reinforcement learning, and completes a six-run experiment matrix over PPO versus GRPO and variance filtering ratios in $\{1.0, 0.5, 0.25\}$ on FrozenLake. The PPO runs directionally reproduce the main RAGEN claims: vanilla StarPO is unstable, moderate StarPO-S filtering mitigates the collapse, and aggressive filtering induces a trade-off. The GRPO runs, however, reverse the expected pattern: vanilla GRPO is the only configuration that consistently learns, reaching final reward $+0.225$, success rate 22.5%, and format compliance 80.8%, while stronger variance filtering reduces performance. Mechanistic analysis suggests that variance-based rollout filtering acts as a critic-data amplifier for PPO, but becomes mostly sample removal for actor-only GRPO because group-relative z-scored advantages are scale-invariant. The report further distinguishes PPO format collapse from healthy GRPO mode convergence and argues that entropy alone is an insufficient echo-trap proxy.

1 Introduction

Large language models are increasingly used as agents that act in external environments rather than as single-turn text generators. Compared with standard RLHF or preference optimization,

multi-turn agent reinforcement learning has three additional difficulties. First, each trajectory contains a sequence of environment turns, and each turn contains a sequence of generated tokens. Second, observations are partial textual renderings of the environment state, so useful behavior depends on context management across turns. Third, rewards are often sparse: most intermediate turns provide little signal, and success or failure may only be known near the end of a trajectory.

RAGEN (Wang et al., 2025) studies this setting through a unified State-Thinking-Actions-Reward Policy Optimization framework, or StarPO. It introduces bi-level generalized advantage estimation (GAE) for turn-level and token-level credit assignment, and StarPO-S, a rollout filtering method that keeps high-reward-variance prompts before policy updates. The paper reports broad experiments across environments such as FrozenLake, Sokoban, CartPole, Bandit, and Math-Countdown, using server-grade hardware, vLLM-backed rollout, fp32 AdamW optimizer states, and multi-seed averaging.

This project asks a narrower but practically important question: how much of this system can be reproduced on a single consumer GPU, and what changes when the algorithm is switched from PPO with a critic to critic-free GRPO? The available machine is deliberately modest for LLM RL: a single NVIDIA RTX 4070 with 8 GB VRAM, running on Windows. A literal reproduction of the full RAGEN hardware and software stack is therefore impossible. The project instead follows a controlled-rewrite strategy: preserve the key algorithmic contracts, implement the missing infrastructure in a modular codebase, and make all

hardware concessions explicit.

1.1 Contributions

The work has two coupled contributions.

C1: engineering reproduction under 8 GB VRAM. The codebase is rewritten into five replaceable modules: environments, agents, RL algorithms, the RAGEN-style training core, and evaluation. On top of this structure, the project implements a consumer-hardware optimization stack that combines AdamW8bit with 32-bit embedding optimizer states, gradient checkpointing, bf16 weights, micro-batching with gradient accumulation, KV-cache restoration for generation, reference-model checkpointing disablement, batched rollout, and alive-only batch shrinking. These changes reduce peak VRAM by roughly 6–8 GB relative to an fp32 AdamW baseline and make 200-step 0.5B-model RL runs feasible on an 8 GB GPU.

C2: algorithmic boundary of variance filtering. The project completes a 2×3 matrix over algorithm $\in \{\text{PPO}, \text{GRPO}\}$ and variance filter ratio $\in \{1.0, 0.5, 0.25\}$. PPO directionally reproduces the RAGEN claims: vanilla PPO collapses, moderate filtering delays and reduces the collapse, and very strong filtering becomes stable but stagnant. GRPO, however, behaves in the opposite direction. Vanilla GRPO is the strongest of all six runs, while filtered GRPO variants are weaker. This suggests that the RAGEN variance filter is not algorithm-agnostic: it stabilizes PPO partly by improving critic training data, but it does not transfer directly to actor-only GRPO.

1.2 Report Organization

Section 2 reviews PPO, GRPO, StarPO, bi-level GAE, and variance-based rollout filtering. Section 3 describes the modular system and the hardware optimization stack. Section 4 defines the six-run experiment matrix, the alignment to the RAGEN baseline, and evaluation metrics. Section 5 presents the results and mechanism analysis. Section 6 discusses limitations and future work, and Section 7 concludes. Appendix A contains detailed tables and training commands.

2 Background

2.1 LLM Agent Reinforcement Learning

In LLM agent RL, the policy π_θ is a language model. At each environment turn, the model receives a textual observation and produces a response containing reasoning text and an action. In this project, outputs are expected to follow a think/answer structure. The environment maps the answer to a discrete action, transitions to the next state, and returns a reward. This differs from single-turn RLHF (Ouyang et al., 2022), preference optimization such as DPO (Rafailov et al., 2023), and reasoning-oriented RL such as DeepSeek-R1 (Guo et al., 2025), because credit assignment spans both environment turns and generated tokens.

The training objective is to maximize expected return,

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right], \quad (1)$$

where τ is a trajectory and r_t is the turn-level reward. The project follows RAGEN in using sparse FrozenLake rewards, a small format penalty, KL regularization against a reference policy, and bi-level advantage estimation.

2.2 PPO

PPO (Schulman et al., 2017) stabilizes policy-gradient learning by optimizing a clipped surrogate. With old policy $\pi_{\theta_{\text{old}}}$, ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (2)$$

and advantage estimate $\hat{A}_t$, the PPO-Clip objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(u_t, v_t)], \quad (3)$$

$$u_t = r_t(\theta) \hat{A}_t, \quad (4)$$

$$v_t = \tilde{r}_t(\theta) \hat{A}_t, \quad (5)$$

$$\tilde{r}_t(\theta) = \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon). \quad (6)$$

The full PPO implementation also includes a value-function loss and entropy regularization,

$$\begin{aligned} L^{\text{PPO}}(\theta, \phi) = & -L^{\text{CLIP}}(\theta) + c_{\text{vf}} L^{\text{VF}}(\phi) \\ & - c_{\text{ent}} \mathcal{H}[\pi_\theta] + \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}}). \end{aligned} \quad (7)$$

This project uses the RAGEN-aligned coefficients $c_{\text{vf}} = 1.0$, $c_{\text{ent}} = 0.001$, $\beta = 0.001$, and clip ratio $\epsilon = 0.2$. PPO uses an actor learning rate of 10^{-6} and a critic learning rate of 10^{-5} .

Bi-level GAE. GAE (Schulman et al., 2016) computes advantages from temporal-difference residuals,

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{T-t} (\gamma \lambda)^l \delta_{t+l}^V. \quad (8)$$

RAGEN adapts this idea to multi-turn LLM agents through bi-level GAE. A turn-level pass estimates advantages at environment boundaries. A token-level pass then distributes the turn-level signal across generated tokens. The current implementation uses $\gamma = 1.0$, $\lambda = 1.0$, and high-level turn discount $\gamma_{\text{turn}} = 0.95$.

Single-mini-batch degeneration. A key implementation boundary appears when all trajectories selected for an update fit into exactly one mini-batch and the number of PPO epochs is one. On the first forward pass, $\pi_\theta = \pi_{\theta_{\text{old}}}$, so $r_t(\theta) = 1$, $\text{clip_frac} = 0$, and approximate KL is zero. The update becomes a KL-regularized single-step policy-gradient update rather than a meaningful PPO-Clip trust-region update. This boundary is central to the interpretation of the 0.25 filter-ratio runs.

2.3 GRPO

Group Relative Policy Optimization (GRPO) (Shao et al., 2024) removes the critic. For each prompt, the old policy samples a group of trajectories with returns $\{R_1, \dots, R_G\}$. Each trajectory receives a group-relative z-scored advantage,

$$\hat{A}_i = \frac{R_i - \mu_g}{\sigma_g + \varepsilon}, \quad (9)$$

$$\mu_g = \frac{1}{G} \sum_{j=1}^G R_j, \quad (10)$$

$$\sigma_g = \sqrt{\frac{1}{G} \sum_{j=1}^G (R_j - \mu_g)^2}. \quad (11)$$

The implementation uses the population standard deviation, matching `torch.std(unbiased=False)`. GRPO then uses a PPO-style clipped surrogate without a value-function term.

The important property is approximate invariance to positive affine transformations of group rewards. If all rewards in a group are transformed as $R'_i = aR_i + b$, $a > 0$, then

$$\hat{A}'_i = \frac{a(R_i - \mu_g)}{a\sigma_g + \varepsilon} \approx \frac{R_i - \mu_g}{\sigma_g + \varepsilon}. \quad (12)$$

Module	Responsibility
envs	Multi-turn environment reset, transition, reward, and action parsing.
agents	LLM loading, single-request generation, and batched generation.
rl_algos	PPO and GRPO losses, advantage computation, and optimizer updates.
ragen_core	Rollout collection, variance filtering, batching, and iteration-level scheduling.
evaluation	Episode-level evaluation metrics and aggregation.

Table 1: Five-module design of RAGEN-Ward.

Thus GRPO primarily uses within-group ranking and relative separation, not the absolute reward scale. This property explains why selecting only high-variance groups does not amplify GRPO’s learning signal in the same way it can help PPO’s critic.

2.4 StarPO and StarPO-S

StarPO organizes multi-turn LLM agent RL around four components: textual state, thinking text, actions, and rewards. Each turn appends format and length instructions, and malformed outputs receive a per-turn format penalty of -0.1 . For FrozenLake, the environment reward is sparse and is not dense-shaped.

StarPO-S adds variance-based rollout filtering. At each training step, P prompts are sampled, and K rollouts are generated for each prompt. For each prompt p , the trainer computes

$$\text{Var}[R \mid p] = \frac{1}{K} \sum_{i=1}^K (R_{p,i} - \bar{R}_p)^2. \quad (13)$$

Prompts are sorted by this variance, and only the top $r \cdot P$ prompts are kept, where r is `variance_filter_ratio`. RAGEN reports that moderate filtering reduces echo-trap behavior in small-model PPO-style training. This project tests whether that conclusion survives an algorithm switch to GRPO.

3 Method and System Design

3.1 Modular Rewrite

The codebase is organized around five modules with explicit contracts:

This design makes the six-run experiment matrix a configuration-level controlled comparison. The PPO and GRPO runs share the same environment, agent interface, rollout machinery, logging

system, evaluation code, and seed. They differ mainly in `rl_algos/ppo.py` versus `rl_algos/grpo.py`, plus the presence or absence of the critic.

3.2 Algorithm Implementation Details

The PPO implementation uses actor-critic parameter groups so that the actor and critic learning rates match the RAGEN baseline. PPO and GRPO both use KL regularization with a frozen reference policy. The format penalty is injected at the turn where the malformed response occurs, rather than being applied once at the end of a trajectory. This keeps the credit-assignment signal local to the failure.

For diagnostics, the implementation records the pre-clipping global gradient norm returned by `clip_grad_norm_`. Reporting the post-clipping value would hide gradient spikes because it is always bounded by `max_grad_norm=1.0`. This detail matters when comparing collapse behavior to RAGEN’s gradient-norm diagnostics.

3.3 Memory Optimization Stack

The memory layer is designed around one constraint: a 0.5B model, a critic, and a reference policy must all fit into 8 GB VRAM during multi-turn RL.

The most subtle fix is KV-cache restoration. In HuggingFace Transformers, enabling gradient checkpointing sets `config.use_cache=False`. That is appropriate for training forward passes, but it also slows autoregressive rollout generation if left unchanged. RAGEN-Ward explicitly restores `use_cache=True` for generation and passes `use_cache=False` only in training forwards. It also switches the actor into evaluation mode between updates so the checkpointing path does not silently disable cache during rollout.

3.4 Throughput Optimization Stack

Because vLLM is not available in the Windows setup used for the experiments, rollout throughput is improved in native PyTorch. The agent layer implements `batched_chat_request`: prompts are left-padded, attention masks isolate padding positions, and a single `generate` call produces multiple completions. The rollout core then advances a batch of trajectories turn by turn. After each environment step, completed trajectories are removed from the next generation batch.

This alive-only shrinking avoids wasting GPU time on trajectories that have already terminated. The combination of batched generation and alive-only shrinking gives a practical $3\times\text{--}5\times$ rollout improvement over naive serial generation while preserving the same per-sequence sampling distribution.

3.5 Data and Logging Infrastructure

Each training step and evaluation point writes metrics to console output, `logs/<exp_name>_metrics.jsonl`, and a full text log. The figures in this report are generated from those logs. This makes the report data path reproducible: the analysis does not rely on hand-edited measurements beyond the plotting and table aggregation scripts.

4 Experimental Setup

4.1 Experiment Matrix

The experiment matrix contains six 200-step runs:

All runs use seed 42, Qwen2.5-0.5B-Instruct (Yang et al., 2024), FrozenLake-v1 (Brockman et al., 2016), sparse reward, randomized maps, 4×4 grids, and the RAGEN-aligned slippery transition probabilities 0.8/0.1/0.1. Each training step samples $P = 8$ prompts and $K = 16$ rollouts per prompt, giving 128 trajectories before filtering.

4.2 Alignment to the RAGEN Baseline

Table 4 summarizes the main aligned and non-aligned settings. The core algorithmic settings are closely aligned, while the hardware and optimizer stack necessarily differ.

The `max_seq_length` detail is important. The code default remains 2048, and the first no-filter run was initially run with that default. Later filter-ratio runs encountered memory pressure after roughly 100 training steps and were run with `--max_seq_length 1536`. Since observed trajectories were usually below 1536 tokens, this is treated as a bounded concession rather than a change that explains the main group ordering.

4.3 Evaluation Protocol and Metrics

Evaluation runs every 20 training steps for 200 episodes. Evaluation environment seeds are sampled by the training process RNG rather than

#	Optimization	Effect and mathematical status
A	AdamW8bit	Saves roughly 3 GB of optimizer-state memory, but introduces block-wise quantization noise.
B	32-bit embedding states	Keeps embedding optimizer states in fp32 to prevent NaNs; no memory saving, but improves stability.
C	Gradient checkpointing	Saves roughly 1 GB of activation memory at the cost of extra recomputation; mathematically equivalent.
D	KV-cache restoration	Re-enables generation cache after checkpointing disables it globally; improves rollout speed by roughly $3\times-10\times$.
E	Reference no-checkpointing	Disables checkpointing for the frozen reference model, whose forward passes run under no-gradient mode.
F	Micro-batch 1, accumulation 32	Preserves effective mini-batch size 32; mathematically equivalent to larger micro-batches but slower.
G	bf16 weights	Saves model-weight memory relative to fp32 and avoids fp16 range issues.
H	Max sequence length	Code default is 2048; later memory-pressure runs override to 1536, truncating only rare long trajectories.
I	Expandable allocator and cache clearing	Reduces effective fragmentation in Windows WDDM edge cases.

Table 2: Memory and stability optimizations used to make training feasible on an 8 GB GPU.

#	Algorithm	Ratio	Purpose
1	PPO	1.0	Vanilla StarPO baseline
2	PPO	0.5	Moderate StarPO-S filtering
3	PPO	0.25	Aggressive StarPO-S filtering
4	GRPO	1.0	Vanilla actor-only baseline
5	GRPO	0.5	Moderate filtering
6	GRPO	0.25	Aggressive filtering

Table 3: Six-run PPO/GRPO by variance-filter-ratio matrix.

fixed to an identical prompt set across steps. Evaluation also respects the agent sampling configuration instead of forcing temperature zero.

The main metrics are:

- **Average reward:** mean cumulative environment reward, excluding the format penalty.
- **Success rate:** strict task completion, using environment success fields when available.
- **Format compliance:** fraction of turns matching the required think/answer output format.
- **Action validity and effectiveness:** turn-level action parsing and usefulness rates.
- **Reward variance:** trajectory-level reward variance, used for echo-trap diagnostics.

The analysis emphasizes average reward, success rate, format compliance, entropy, KL penalty, gradient norm, and the number of gradient steps per training iteration.

5 Results and Analysis

5.1 Overall Scorecard

Table 5 gives the final state of all six runs. The central empirical fact is simple: the PPO column directionally supports RAGEN’s StarPO-S story, while the GRPO column reverses it.

Figure 1 shows the full reward trajectories. PPO with no filtering improves briefly and then collapses after step 100, ending at -0.654 . PPO with ratio 0.5 reaches its best point at step 120 and remains much better than vanilla at the end. PPO with ratio 0.25 is stable but nearly stagnant. GRPO behaves differently: vanilla GRPO steadily improves to $+0.225$, while both filtered GRPO variants remain near the negative-reward plateau.

5.2 PPO Directionally Reproduces RAGEN

The PPO results support three RAGEN-style claims at the level of trend and mechanism.

P1: vanilla instability. PPO with filter ratio 1.0 deteriorates from -0.124 at step 20 to -0.654 at step 200, a roughly $5.3\times$ worsening relative to the initial magnitude. This reproduces the instability direction in the RAGEN 0.5B sparse-reward setting.

P2: moderate filtering mitigates collapse.

PPO with ratio 0.5 reaches -0.054 at step 120, while vanilla PPO is already at -0.516 at the same point. At the final step, the moderate filter is still 0.469 reward points better than vanilla. Interpreted as reduction of the vanilla loss magnitude, this recovers roughly 71% of the damage.

Setting	RAGEN-Ward	RAGEN baseline	Status
Model	Qwen2.5-0.5B-Instruct	Qwen2.5-0.5B-Instruct	aligned
Environment	FrozenLake-v1	FrozenLake-v1	aligned
Training steps	200	200	aligned
Prompt batch / rollouts	8/16	8/16	aligned
KL coefficient	0.001	0.001	aligned
Entropy coefficient	0.001	0.001	aligned
PPO actor / critic LR	$10^{-6}/10^{-5}$	$10^{-6}/10^{-5}$	aligned
Clip ratio	0.2	0.2	aligned
GAE settings	$\gamma = 1.0, \lambda = 1.0, \gamma_{\text{turn}} = 0.95$	same	aligned
Optimizer	AdamW8bit plus fp32 embeddings	fp32 AdamW	hardware concession
Rollout engine	Native PyTorch batched rollout	vLLM-backed rollout	hardware/software concession
Micro-batch / accumulation	$1 \times 32 = 32$	$4 \times 8 = 32$	mathematically equivalent
Max sequence length	default 2048; some later runs use 1536	3600	bounded concession
Seeds	one seed, 42	multi-seed average	major limitation
Eval episodes	200	500	higher evaluation variance

Table 4: Parameter alignment and hardware concessions relative to the RAGEN baseline.

#	Run	Algorithm	Filter	Reward	Success	Format
1	PPO vanilla	PPO	1.0	-0.654	0.000	0.000
2	PPO StarPO-S moderate	PPO	0.5	-0.185	0.000	0.000
3	PPO StarPO-S aggressive	PPO	0.25	-0.060	0.005	0.050
4	GRPO vanilla	GRPO	1.0	+0.225	0.225	0.808
5	GRPO filtered moderate	GRPO	0.5	-0.079	0.005	0.369
6	GRPO filtered aggressive	GRPO	0.25	-0.072	0.020	0.363

Table 5: Final evaluation metrics at step 200. GRPO vanilla is the only run with sustained positive reward, non-trivial success, and high format compliance.

P3: filtering is a trade-off. PPO with ratio 0.25 is more stable than the other two PPO runs but does not show meaningful learning. The apparent final advantage of the 0.25 run should not be read as a clean StarPO-S win, because this configuration triggers the single-mini-batch degeneration discussed below. The healthier PPO learning window is the ratio 0.5 run before it later deteriorates.

5.3 PPO Collapse Is Format Collapse, Not Classic Echo Trap

The vanilla PPO failure does not match RAGEN’s classic echo-trap signature. RAGEN echo trap is characterized by repeated invalid modes and decreasing entropy. Here, PPO vanilla enters a format collapse: reward falls, format compliance reaches zero, KL spikes sharply, and entropy increases rather than decreases.

Figures 3 and 4 make the distinction visible. PPO vanilla reaches the highest entropy among the six runs, while format compliance falls to zero. The run also shows an extreme KL spike near

the collapse boundary. This points to a failure mode in which the policy moves away from the reference model into high-entropy malformed output, rather than converging to a deterministic repeated phrase.

5.4 Filter Ratio 0.25 Degenerates to Single-Step Updates

With $P = 8$, $K = 16$, and effective mini-batch size 32, the number of gradient steps per training iteration is

$$n_{\text{grad}} = \left\lceil \frac{r \cdot P \cdot K}{32} \right\rceil. \quad (14)$$

Thus ratios 1.0, 0.5, and 0.25 produce 4, 2, and 1 gradient steps respectively.

At ratio 0.25, the filtered buffer contains exactly 32 trajectories, equal to one mini-batch. With one epoch, the first and only update sees current log probabilities equal to old log probabilities, so the PPO ratio is one, clipping never activates, and approximate KL is zero. This degeneration affects both PPO and GRPO because GRPO reuses the PPO-style clipped surrogate. The stability

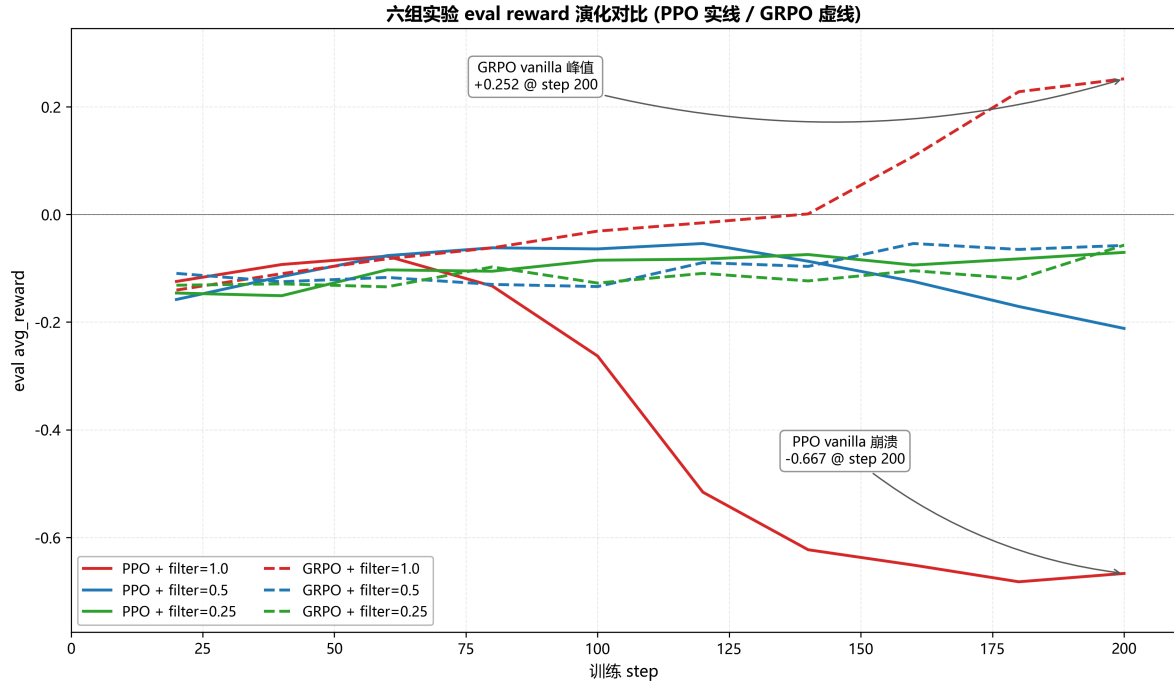


Figure 1: Evaluation reward over all six runs. PPO exhibits the expected filtering trade-off direction, while GRPO performs best with no variance filtering.

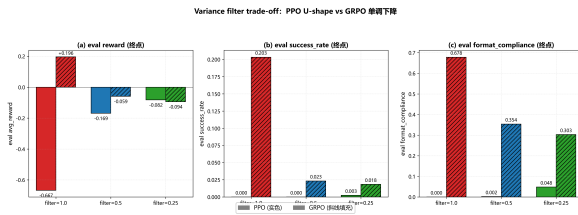


Figure 2: Final-reward view of the filter trade-off. PPO and GRPO respond to filtering in opposite directions.

of the 0.25 runs is therefore better interpreted as conservative single-step REINFORCE-like behavior (Williams, 1992), not as proof that aggressive variance filtering is intrinsically superior.

5.5 GRPO Reverses the StarPO-S Pattern

The GRPO runs are the main extension beyond a straightforward reproduction. GRPO vanilla is the only run that reaches positive reward, non-trivial success, and high format compliance. Figures 6 and 7 show that its reward improvement is not a metric artifact: the agent both solves the task more often and preserves the required output structure.

At the claim level, the GRPO column is opposite to the PPO column:

- Vanilla GRPO is not unstable; it is the

strongest run.

- Filtering does not repair vanilla GRPO; it makes GRPO worse.
- The best GRPO filter ratio is 1.0, not 0.5.

5.6 Why Filtering Helps PPO but Hurts GRPO

The mechanism has three layers.

Critic-data amplification exists only in PPO.

For PPO, high-variance prompts provide useful value-function targets: the same prompt can produce trajectories with different returns, so the critic receives informative state-return contrasts. A better critic improves advantage estimates and stabilizes the actor update. In this sense, variance filtering acts as data selection for the critic. GRPO has no critic. Its advantage is a within-group z-score. Once the group has nonzero reward variation, scaling the reward spread does not proportionally scale the advantage. Filtering high-variance groups therefore does not provide the same signal amplification; it mostly removes samples.

Sample removal is more costly in GRPO. For PPO, the benefit of cleaner critic data can partially offset the harm of fewer samples, producing a sweet spot. For GRPO, the benefit side is

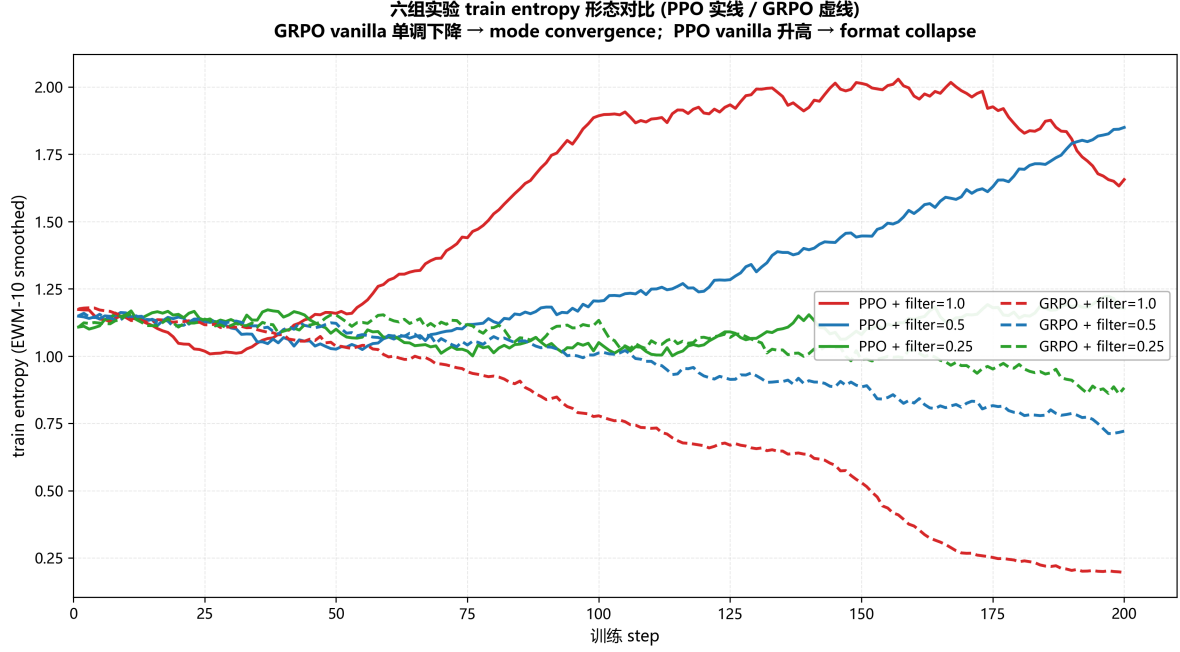


Figure 3: Training entropy. PPO vanilla moves toward higher entropy during collapse, while GRPO vanilla decreases entropy while improving reward and format compliance.

much weaker. Reducing samples increases gradient variance and reduces the number of update mini-batches without providing a compensating critic-training improvement.

The 0.25 boundary is shared. Both algorithms degenerate at ratio 0.25 under the single-GPU mini-batch configuration. This explains why both aggressive-filter runs appear stable but stagnant.

5.7 Echo-Trap Proxy Needs Reward and Format

Entropy decrease alone is not enough to diagnose echo trap. Figure 9 shows that PPO vanilla and GRPO vanilla are opposite across all four key dimensions. PPO vanilla has reward down, entropy up, format down, and success zero. GRPO vanilla has reward up, entropy down, format up, and success up.

Figure 10 motivates a stronger diagnostic: echo trap should be treated as a joint pattern involving entropy, reward, and format compliance. Low entropy with rising reward and rising format compliance is healthy mode convergence. Low entropy with stagnant or falling reward is a candidate echo trap. High entropy with format collapse is a different failure mode.

5.8 Quantization Noise and Algorithm Scale

The PPO runs have pre-clip gradient norms in the hundreds or thousands, while GRPO remains around 0.05–0.16. This is consistent with the different advantage scales: PPO’s bi-level GAE can produce larger token-level weights, while GRPO z-scores are naturally bounded to a small range.

Because AdamW8bit introduces block-wise optimizer-state quantization noise (Dettmers et al., 2022), PPO’s larger update scale and critic coupling may be more sensitive to numerical drift. GRPO provides a useful controlled contrast: it uses the same hardware concessions and optimizer family but avoids the critic and uses smaller advantages. Its strong vanilla performance suggests that part of PPO’s collapse is an algorithm-by-hardware interaction rather than a pure reproduction of RAGEN’s original echo-trap dynamics.

6 Limitations and Future Work

6.1 Limitations

Single seed. Each configuration is run only once with seed 42. RL training is seed-sensitive, so absolute reward values and detailed collapse timing should not be overinterpreted. The strongest conclusions rely on relative gaps and

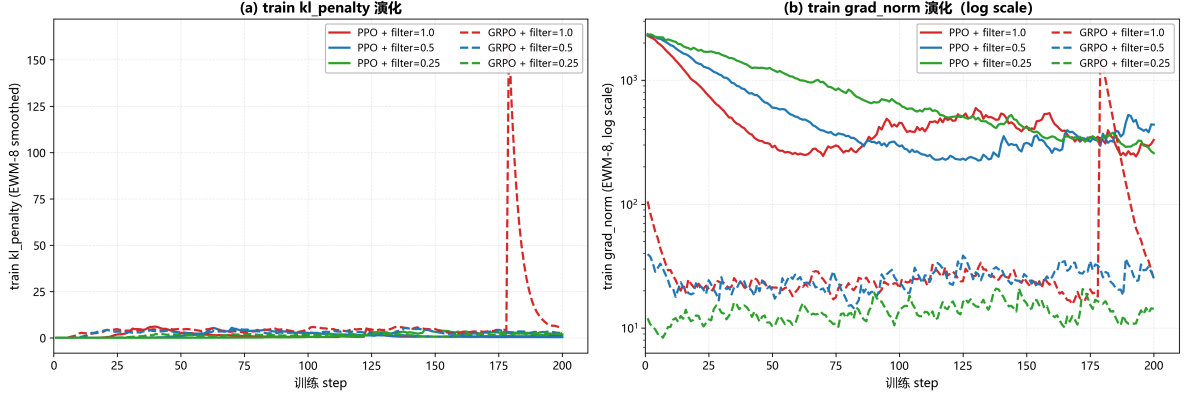


Figure 4: KL penalty and pre-clip gradient norm. PPO runs have much larger gradient norms than GRPO runs, and PPO vanilla shows a large KL spike near the collapse boundary.

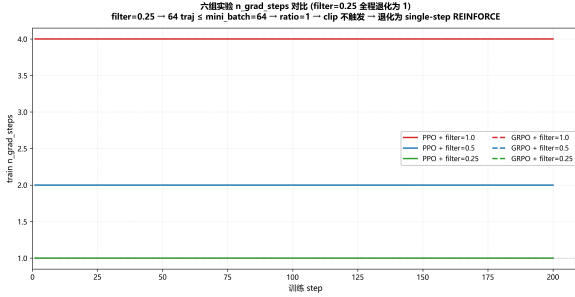


Figure 5: The number of optimizer steps is determined by filter ratio under the fixed effective mini-batch size. Ratio 0.25 gives exactly one update step.

curve shapes: for example, GRPO vanilla is roughly 0.30 reward points above the filtered GRPO variants at the end, a gap larger than the expected seed-level noise estimated in the original analysis.

One trained environment. The full training experiments are all on FrozenLake. Other RAGEN-style environments are implemented in the codebase and have smoke-test or wrapper-check paths, but they are not trained in the six-run matrix. Therefore the mechanism claim should be read as applying to the observed setting: multi-turn interaction, sparse rewards, randomized FrozenLake maps, a 0.5B model, one seed, and consumer hardware.

No fp32 AdamW baseline. The hypothesis that AdamW8bit partly contributes to PPO instability is supported indirectly by the GRPO contrast, not by a direct fp32 reproduction. A decisive test would require fp32 AdamW with multiple seeds,

which exceeds the available 8 GB VRAM budget.

Filter and sample count are coupled. Variance filtering changes both which prompts are selected and how many trajectories enter each update. Under the available hardware, the project does not run equal-sample controls in which filtered configurations generate more rollouts to compensate for dropped samples. This limitation is especially relevant for GRPO, where the interpretation is that filtering mostly removes samples.

Optimization-stack ablations are not isolated. The engineering stack is enabled as a whole. The report classifies the expected mathematical impact of each concession, but it does not run ablations for each memory or throughput optimization. The most consequential unresolved concession is AdamW8bit; most other changes are implementation-level equivalences or bounded truncation effects.

6.2 Future Work

The most valuable next experiments are straightforward.

Multi-seed PPO and GRPO. Repeating the six-run matrix over at least three seeds would separate systematic algorithmic effects from seed-specific collapse modes. This is the first experiment needed before making strong general claims.

Cross-environment training. The codebase is prepared for Bandit, CartPole, Sokoban, Math-Countdown, and FrozenLake through the same

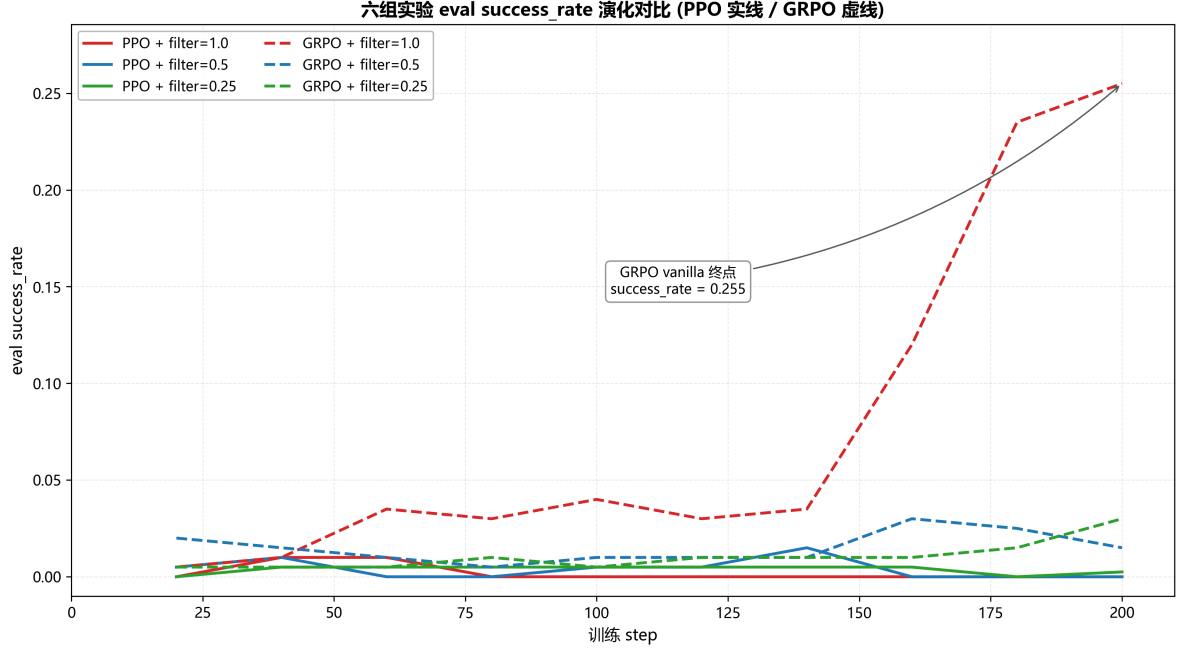


Figure 6: Evaluation success rate. GRPO vanilla is the only run with substantial task success at the end of training.

environment interface. Sokoban is the most relevant next environment because it stresses multi-step planning. Math-Countdown is the most relevant for reasoning-RL comparison to GRPO-style work.

fp32 or higher-memory optimizer control. A server-side run with fp32 AdamW, or at least a larger-memory GPU allowing a less aggressive optimizer concession, would directly test the quantization-noise hypothesis.

Finer filter sweep. Runs with filter ratio 0.75 would clarify whether GRPO performance degrades monotonically as filtering becomes stronger, and whether PPO’s practical sweet spot is closer to 0.5 or another intermediate value under consumer-hardware constraints.

Avoiding 0.25 degeneration. The aggressive-filter runs could be repeated with `ppo_epochs=2` or a smaller effective mini-batch so that PPO-Clip and GRPO-Clip have more than one update opportunity. This would separate true aggressive-filter behavior from the single-mini-batch boundary.

Linux and vLLM migration. Moving the same modular codebase to Linux with vLLM would reduce rollout time and make multi-seed, multi-environment sweeps more affordable.

7 Conclusion

RAGEN-Ward shows that a meaningful subset of RAGEN-style multi-turn LLM agent RL can be reproduced on a single 8 GB consumer GPU, provided the system is aggressively engineered for memory and rollout throughput. The modular rewrite and optimization stack make 200-step 0.5B PPO and GRPO training feasible under constraints that are far below the original server-grade setup.

Scientifically, the six-run matrix gives a two-sided result. PPO directionally reproduces RAGEN’s claims: vanilla StarPO is unstable, moderate variance filtering mitigates the failure, and overly aggressive filtering becomes stagnant. GRPO reveals the boundary of that mechanism. Without a critic, variance filtering no longer acts as critic-data selection; because GRPO advantages are group-relative z-scores, filtering mostly reduces sample count and update opportunities. In the observed setting, vanilla GRPO is therefore the strongest configuration.

The practical recommendation is accordingly narrow but useful: under 8 GB VRAM constraints for sparse multi-turn LLM agent RL, GRPO with no variance filtering should be considered before directly transferring PPO-oriented StarPO-S filtering. More broadly, echo-trap diagnosis should not rely on entropy alone. Reward, format com-

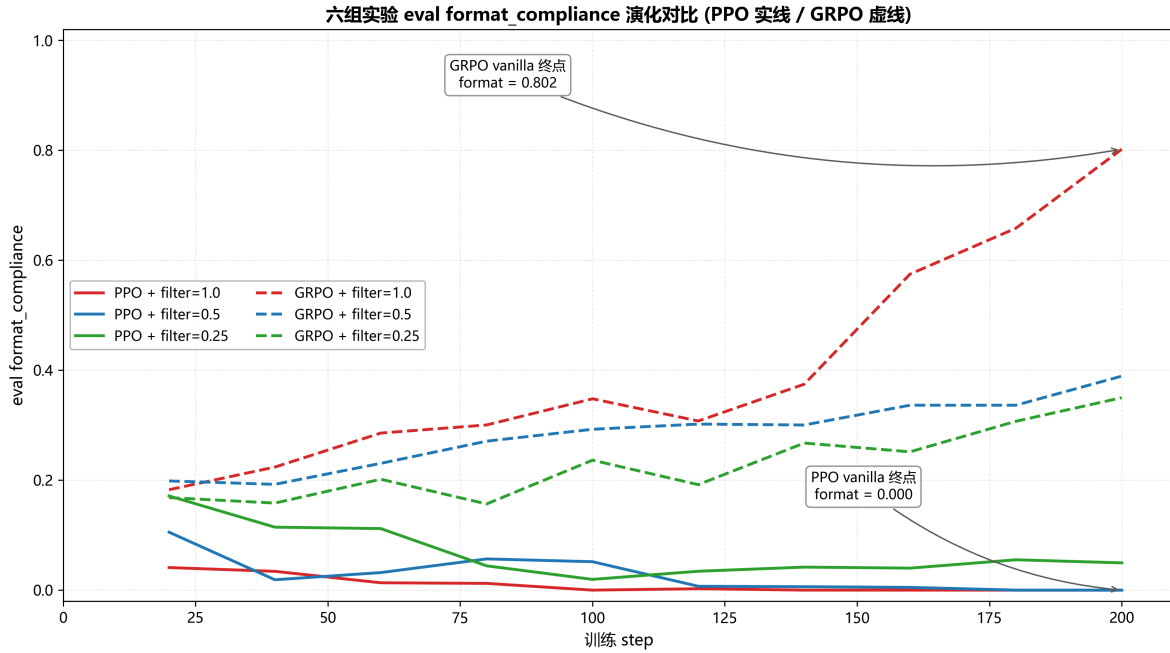


Figure 7: Format compliance. GRPO vanilla rises to approximately 0.81, while PPO variants and filtered GRPO variants remain much weaker.

pliance, and entropy must be interpreted jointly to distinguish unhealthy collapse from healthy mode convergence.

References

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. 2016. [OpenAI Gym](#). *Preprint*, arXiv:1606.01540.

Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 2022. [8-bit optimizers via block-wise quantization](#). In *International Conference on Learning Representations*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shitong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. [DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning](#). *Preprint*, arXiv:2501.12948.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. [Training language models to follow instructions with human feedback](#). In *Advances in Neural Information Processing Systems*.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea

Finn. 2023. [Direct preference optimization: Your language model is secretly a reward model](#). In *Advances in Neural Information Processing Systems*.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. 2016. [High-dimensional continuous control using generalized advantage estimation](#). In *International Conference on Learning Representations*.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. [Proximal policy optimization algorithms](#). *Preprint*, arXiv:1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. [DeepSeekMath: Pushing the limits of mathematical reasoning in open language models](#). *Preprint*, arXiv:2402.03300.

Z. Wang, K. Wang, Q. Wang, P. Zhang, L. Li, Z. Yang, K. Yu, M. N. Nguyen, L. Liu, E. Gottlieb, M. Lam, Y. Lu, K. Cho, J. Wu, L. Fei-Fei, L. Wang, Y. Choi, and M. Li. 2025. [RAGEN: Understanding self-evolution in LLM agents via multi-turn reinforcement learning](#). *Preprint*, arXiv:2504.20073.

Ronald J. Williams. 1992. [Simple statistical gradient-following algorithms for connectionist reinforcement learning](#). *Machine Learning*, 8(3–4):229–256.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, and 1 others. 2024. [Qwen2.5 technical report](#). *Preprint*, arXiv:2412.15115.

六组实验 eval 五指标多面板对比

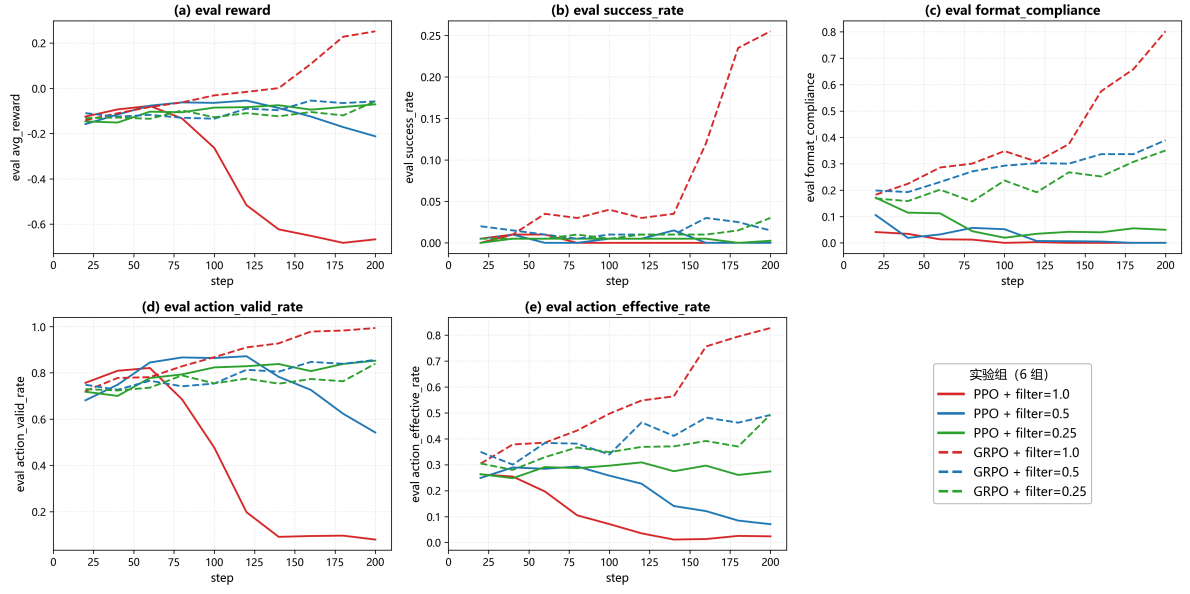


Figure 8: Five evaluation metrics. GRPO vanilla is strongest or tied strongest across the main evaluation dimensions.

A Detailed Evaluation Tables

A.1 Average Reward

A.2 Success Rate

A.3 Format Compliance

A.4 Training Metric Snapshot

PPO vanilla vs GRPO vanilla: 双线叙事核心对比 (filter=1.0)

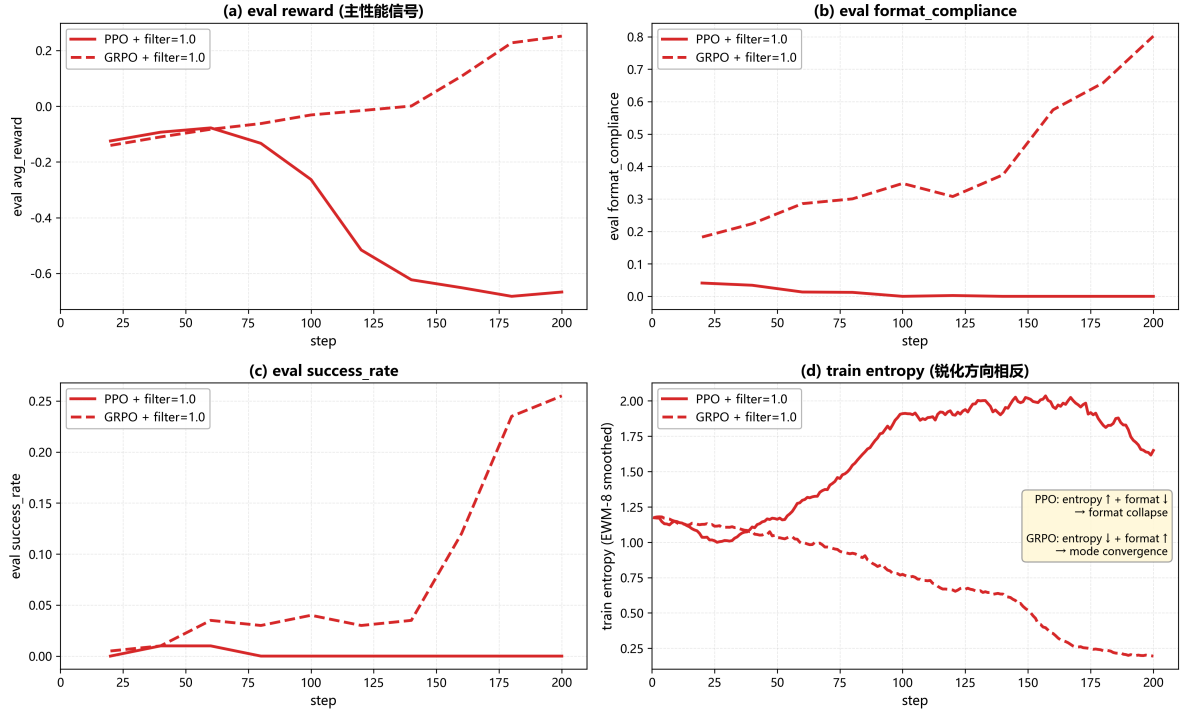


Figure 9: Head-to-head comparison of PPO vanilla and GRPO vanilla. The two runs move in opposite directions across reward, entropy, format compliance, and success rate.

Echo trap proxy 形态对比: format collapse vs mode convergence

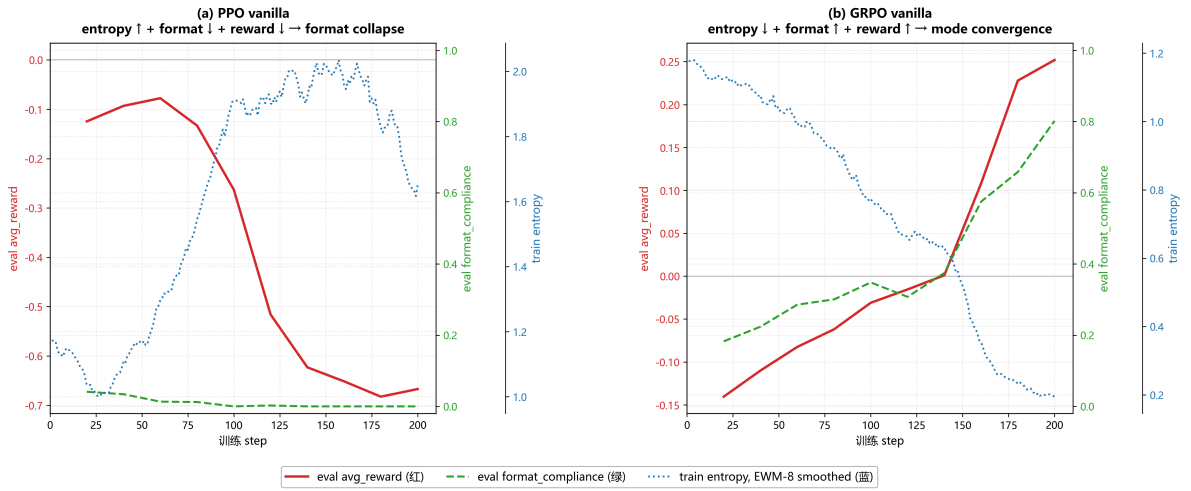


Figure 10: Joint echo-trap proxy. Entropy alone is ambiguous; reward and format compliance separate unhealthy collapse from healthy mode convergence.

Step	PPO 1.0	PPO 0.5	PPO 0.25	GRPO 1.0	GRPO 0.5	GRPO 0.25
20	-0.124	-0.158	-0.146	-0.038	-0.105	-0.140
40	-0.093	-0.116	-0.151	-0.026	-0.103	-0.131
60	-0.078	-0.077	-0.103	-0.045	-0.078	-0.105
80	-0.133	-0.062	-0.106	+0.022	-0.108	-0.110
100	-0.263	-0.064	-0.085	-0.013	-0.111	-0.073
120	-0.516	-0.054	-0.083	+0.012	-0.064	-0.090
140	-0.623	-0.087	-0.075	+0.029	-0.046	-0.072
160	-0.652	-0.125	-0.094	+0.139	-0.026	-0.078
180	-0.683	-0.171	-0.083	+0.228	-0.075	-0.026
200	-0.654	-0.185	-0.060	+0.225	-0.079	-0.072

Table 6: Evaluation average reward over training.

Step	PPO 1.0	PPO 0.5	PPO 0.25	GRPO 1.0	GRPO 0.5	GRPO 0.25
20	0.000	0.000	0.000	0.080	0.005	0.000
40	0.000	0.000	0.000	0.085	0.005	0.000
60	0.000	0.000	0.000	0.075	0.020	0.005
80	0.000	0.000	0.000	0.140	0.005	0.005
100	0.000	0.000	0.000	0.115	0.005	0.020
120	0.000	0.005	0.005	0.130	0.025	0.010
140	0.000	0.000	0.005	0.140	0.025	0.025
160	0.000	0.000	0.005	0.215	0.040	0.020
180	0.000	0.000	0.005	0.245	0.005	0.030
200	0.000	0.000	0.005	0.225	0.005	0.020

Table 7: Evaluation success rate over training.

Step	PPO 1.0	PPO 0.5	PPO 0.25	GRPO 1.0	GRPO 0.5	GRPO 0.25
20	0.220	0.230	0.205	0.265	0.235	0.220
40	0.180	0.260	0.220	0.305	0.290	0.265
60	0.200	0.275	0.270	0.345	0.310	0.330
80	0.155	0.305	0.305	0.450	0.320	0.355
100	0.085	0.305	0.330	0.430	0.340	0.395
120	0.030	0.285	0.345	0.495	0.350	0.425
140	0.005	0.230	0.345	0.625	0.345	0.420
160	0.005	0.150	0.290	0.685	0.335	0.380
180	0.000	0.080	0.180	0.795	0.330	0.385
200	0.000	0.000	0.050	0.808	0.369	0.363

Table 8: Evaluation format compliance over training.

Metric	PPO 1.0	PPO 0.5	PPO 0.25	GRPO 1.0	GRPO 0.5	GRPO 0.25
n_grad_steps	4	2	1	4	2	1
Entropy at step 200	~ 1.93	~ 1.82	~ 1.18	~ 0.19	~ 0.85	~ 1.16
Grad norm at step 200	~ 120	~ 80	~ 95	~ 0.16	~ 0.10	~ 0.05
KL penalty at step 200	~ 3.0	~ 1.5	~ 0.8	< 0.01	< 0.01	< 0.01
Clip behavior	active	active	zero	active	active	zero
Approx. KL	positive	positive	zero	positive	positive	zero

Table 9: Training metric snapshot used to diagnose degeneration and optimizer-scale differences.

B Training Commands

The current code default for `max_seq_length` is 2048. The commands below record the reproducible later-run form with an explicit 1536 override. The first no-filter run was originally run with the 2048 default; later runs used the 1536 override because of memory pressure after roughly 100 training iterations.

```
python scripts/train.py \\  
  --algo ppo \\  
  --variance_filter_ratio 1.0 \\  
  --max_seq_length 1536 \\  
  --exp_name ragen_baseline_0.5B_ppo_nofilter
```

```
python scripts/train.py \\  
  --algo ppo \\  
  --variance_filter_ratio 0.5 \\  
  --max_seq_length 1536 \\  
  --exp_name ragen_baseline_0.5B_ppo_filter05
```

```
python scripts/train.py \\  
  --algo ppo \\  
  --variance_filter_ratio 0.25 \\  
  --max_seq_length 1536 \\  
  --exp_name ragen_baseline_0.5B_ppo_filter025
```

```
python scripts/train.py \\  
  --algo grpo \\  
  --variance_filter_ratio 1.0 \\  
  --max_seq_length 1536 \\  
  --exp_name ragen_baseline_0.5B_grpo_nofilter
```

```
python scripts/train.py \\  
  --algo grpo \\  
  --variance_filter_ratio 0.5 \\  
  --max_seq_length 1536 \\  
  --exp_name ragen_baseline_0.5B_grpo_filter05
```

```
python scripts/train.py \\  
  --algo grpo \\  
  --variance_filter_ratio 0.25 \\  
  --max_seq_length 1536 \\  
  --exp_name ragen_baseline_0.5B_grpo_filter025
```

The Windows environment variable used for memory-fragmentation mitigation is:

```
$env:PYTORCH_CUDA_ALLOC_CONF = "expandable_segments:True"
```